

Tesis

Mariana Belén Cruz Rodríguez Yofre Hernán García Gómez

Tabla de contenidos

1	Introducción	4
1.1	CNN adaptada a la clasificación de glifos	4
1.2	Representación de imágenes	4
1.3	Estructura funcional de la red	5
1.4	Parámetros del modelo	5
2	Operación de Convolución	6
2.1	Caso unicanal	6
2.2	Interpretación como producto interno	6
3	Convolución Multicanal	8
3.1	Interpretación tensorial	8
4	Múltiples Filtros (Mapas de Características)	9
5	Interpretación de los kernels	10
5.1	Propiedad de compartición de parámetros	10
6	Stride y Padding	11
6.1	Stride	11
6.2	Padding	11
7	Función de activación	12
7.0.1	Definición	12
7.0.2	No linealidad	12
7.0.3	Ejemplos de funciones de activación	13
7.0.4	Propiedades fundamentales	13
7.0.5	Formulación funcional	14
8	Función de Activación (Resumen)	15
9	Pooling	16
9.1	Max Pooling	16
9.2	Average Pooling	16
10	Capa Totalmente Conectada	17
11	Función de Pérdida	18
11.1	Softmax	18
11.2	Entropía Cruzada	18

12 Entrenamiento	19
13 Propiedades Fundamentales	20
13.1 Conectividad Local	20
13.2 Compartición de Parámetros	20
13.3 Invarianza Traslacional	20
14 Flujo de una CNN	21
15 Interpretación Matemática	22

1 Introducción

En este trabajo se aborda el problema de **reconocimiento automático de glifos del alfabeto náhuatl** mediante el uso de redes neuronales convolucionales (*Convolutional Neural Networks*, CNN). Este problema puede formularse como una tarea de clasificación de imágenes, en la cual cada glifo es representado mediante una estructura matricial que codifica su información visual.

Los glifos presentan variaciones en forma, trazo y estilo, lo que dificulta su identificación mediante métodos tradicionales. En este contexto, las CNN resultan particularmente adecuadas debido a su capacidad para extraer características espaciales relevantes directamente a partir de los datos, tales como bordes, curvas y patrones geométricos distintivos.

1.1. CNN adaptada a la clasificación de glifos

Desde un punto de vista formal, una CNN puede entenderse como una función parametrizada

$$f(\cdot, \theta) : \mathcal{X} \subseteq \mathbb{R}^{H \times W \times C} \longrightarrow \mathbb{R}^k \quad (1.1)$$

donde:

- $\mathcal{X}$ es el espacio de entrada (por ejemplo, imágenes),
- H representa la **altura** (*height*) de la imagen, es decir, el número de filas,
- W representa el **ancho** (*width*), es decir, el número de columnas,
- C representa el **número de canales** (*channels*), que corresponde a la cantidad de valores asociados a cada píxel,
- k es la dimensión del espacio de salida (por ejemplo, número de clases en clasificación),
- θ denota el conjunto total de parámetros entrenables del modelo.

1.2. Representación de imágenes

Una imagen se modela matemáticamente como un tensor de orden 3:

$$X \in \mathbb{R}^{H \times W \times C} \quad (1.2)$$

Esto significa que:

- Para cada posición espacial (i, j) con $1 \leq i \leq H$, $1 \leq j \leq W$,
- el valor $X(i, j)$ no es escalar, sino un vector en $\mathbb{R}^C$:

$$X(i, j) = (x_1, x_2, \dots, x_C) \quad (1.3)$$

1.3. Estructura funcional de la red

Una CNN se construye como una composición de funciones:

$$f(x; \theta) = f^{(L)} \circ f^{(L-1)} \circ \dots \circ f^{(1)}(x) \quad (1.4)$$

donde:

- $L \in \mathbb{N}$ es el número total de capas de la red,
- cada $f^{(l)}$ es una función asociada a la capa l -ésima.

1.4. Parámetros del modelo

El conjunto de parámetros θ contiene todos los elementos ajustables del modelo:

$$\theta = \theta^{(1)}, \theta^{(2)}, \dots, \theta^{(L)} \quad (1.5)$$

En una CNN, estos parámetros incluyen principalmente:

- **kernels** (filtros convolucionales),
- **sesgos** (bias) asociados a cada filtro.

Estos parámetros determinan completamente la forma de la función $f(x; \theta)$ y son aprendidos a partir de los datos mediante optimización.

2 Operación de Convolución

La operación fundamental en una CNN es la **convolución discreta** (en la práctica, correlación cruzada).

2.1. Caso unicanal

Sea una imagen:

$$X \in \mathbb{R}^{H \times W} \quad (2.1)$$

y un kernel (o filtro):

$$K \in \mathbb{R}^{k_h \times k_w} \quad (2.2)$$

donde:

- k_h : altura del kernel,
- k_w : ancho del kernel.

La salida Y se define como:

$$Y(i, j) = \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} K(u, v), X(i+u, j+v) \quad (2.3)$$

2.2. Interpretación como producto interno

Sea $X_{(i,j)}$ la submatriz de X de tamaño $k_h \times k_w$:

$$X_{(i,j)} = X(i+u, j+v)_{u,v} \quad (2.4)$$

Entonces:

$$Y(i, j) = \langle K, X_{(i,j)} \rangle \quad (2.5)$$

donde $\langle \cdot, \cdot \rangle$ denota el producto interno en $\mathbb{R}^{k_h \times k_w}$.

Esto implica que la convolución mide la **similitud entre el kernel y una región local de la imagen**.

3 Convolución Multicanal

En el caso general, la entrada es:

$$X \in \mathbb{R}^{H \times W \times C} \quad (3.1)$$

y el kernel:

$$K \in \mathbb{R}^{k_h \times k_w \times C} \quad (3.2)$$

La convolución se define como:

$$Y(i, j) = \sum_{c=1}^C \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} K(u, v, c), X(i+u, j+v, c) \quad (3.3)$$

3.1. Interpretación tensorial

Si se vectoriza cada ventana:

$$X_{(i,j)} \in \mathbb{R}^{k_h k_w C}, \quad K \in \mathbb{R}^{k_h k_w C} \quad (3.4)$$

entonces:

$$Y(i, j) = K^\top X_{(i,j)} \quad (3.5)$$

Esto muestra que la convolución es un operador lineal local.

4 Múltiples Filtros (Mapas de Características)

En una CNN no se utiliza un solo kernel, sino un conjunto de filtros:

$$K^{(f)} \in \mathbb{R}^{k_h \times k_w \times C}, \quad f = 1, \dots, F \quad (4.1)$$

Cada filtro produce un mapa de activación distinto:

$$Y_f(i, j) = \sum_{c=1}^C \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} K^{(f)}(u, v, c) X(i+u, j+v, c) \quad (4.2)$$

Por tanto, la salida total es un tensor:

$$Y \in \mathbb{R}^{H' \times W' \times F} \quad (4.3)$$

donde:

- H' y W' dependen del tamaño del kernel, el padding y el stride,
- F es el número de filtros, es decir, el número de **canales de salida**.

5 Interpretación de los kernels

Cada kernel puede interpretarse como un **detector de patrones locales**.

Formalmente:

$$Y_f(i, j) = \langle K^{(f)}, X_{(i, j)} \rangle \quad (5.1)$$

Por tanto:

- Si $Y_f(i, j)$ es grande, entonces la región $X_{(i, j)}$ es similar al patrón definido por $K^{(f)}$.
- Si es pequeño o negativo, no hay coincidencia significativa.

5.1. Propiedad de compartición de parámetros

Un aspecto fundamental es que el mismo kernel $K^{(f)}$ se aplica en todas las posiciones (i, j) :

$$K^{(f)}(u, v, c) \text{ es independiente de } (i, j) \quad (5.2)$$

Esto implica:

- reducción significativa del número de parámetros,
- capacidad de detectar patrones independientemente de su posición (invarianza traslacional).

6 Stride y Padding

6.1. Stride

El stride s controla el desplazamiento del kernel. Las dimensiones de salida son:

$$H' = \left\lfloor \frac{H - k_h + 2p}{s} \right\rfloor + 1, \quad W' = \left\lfloor \frac{W - k_w + 2p}{s} \right\rfloor + 1 \quad (6.1)$$

6.2. Padding

El padding es una operación que consiste en añadir valores (generalmente ceros) alrededor de la entrada antes de aplicar la convolución. Esto se hace para controlar el tamaño de la salida y preservar información en los bordes.

$$X_{\text{pad}} \in \mathbb{R}^{(H+2p) \times (W+2p) \times C} \quad (6.2)$$

El padding puede interpretarse como una extensión de la función de entrada X fuera de su dominio original. Formalmente, se define la extensión $\tilde{X}$ como:

$$\tilde{X}(i, j) = \begin{cases} X(i, j), & \text{si } (i, j) \text{ pertenece al dominio original,} \\ 0, & \text{en otro caso} \end{cases} \quad (6.3)$$

Esta extensión corresponde a una extensión por cero del dominio de la función.

En una capa convolucional, el padding se aplica antes de la convolución. Por lo tanto, la operación completa de una capa convolucional con padding se puede expresar como:

$$Y = \sigma((\text{pad}(X)) \star K + b) \quad (6.4)$$

donde:

- X es el tensor de entrada
- $\text{pad}(X)$ es la entrada extendida mediante padding
- $\star$ denota la operación de correlación cruzada
- K es el kernel o filtro
- b es el sesgo
- σ es la función de activación

7 Función de activación

Una **función de activación** es una aplicación no lineal que se introduce después de una transformación afín, con el objetivo de dotar al modelo de la capacidad de aproximar funciones no lineales.

7.0.1. Definición

Sea $x \in \mathbb{R}^n$ un vector de entrada, $w \in \mathbb{R}^n$ un vector de pesos y $b \in \mathbb{R}$ un sesgo. Se define la preactivación como:

$$z = w^T x + b \quad (7.1)$$

La salida de la neurona está dada por:

$$y = \sigma(z) \quad (7.2)$$

donde $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ es la función de activación.

7.0.2. No linealidad

Considérese una red neuronal profunda compuesta únicamente por transformaciones afines. En tal caso, la composición de dichas transformaciones satisface:

$$f(x) = W_L W_{L-1} \cdots W_1 x + b' \quad (7.3)$$

lo cual sigue siendo una transformación afín. Por consiguiente, sin la introducción de funciones de activación no lineales, la red carece de capacidad expresiva para modelar relaciones no lineales.

En una red neuronal convolucional (CNN), la salida de una capa se expresa como:

$$Y = \sigma(X \star K + b) \quad (7.4)$$

donde:

- X es el tensor de entrada
- $\star$ denota la operación de correlación cruzada
- K es el conjunto de filtros (kernels)
- b es el sesgo
- σ se aplica elemento a elemento

7.0.3. Ejemplos de funciones de activación

Algunas de las funciones de activación más utilizadas son:

- **ReLU (Rectified Linear Unit)**

$$\sigma(x) = \max(0, x) \quad (7.5)$$

- **Sigmoide**

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (7.6)$$

- **Tangente hiperbólica**

$$\sigma(x) = \tanh(x) \quad (7.7)$$

7.0.4. Propiedades fundamentales

Las funciones de activación empleadas en redes neuronales satisfacen típicamente:

- **No linealidad**, necesaria para incrementar la capacidad de representación del modelo
- **Aplicación puntual (element-wise)**, preservando la estructura espacial en CNN
- **Diferenciabilidad** (al menos casi en todas partes), requisito esencial para métodos de optimización basados en gradiente

7.0.5. Formulación funcional

Desde una perspectiva funcional, una red neuronal profunda puede interpretarse como la composición de aplicaciones de la forma:

$$f_{\theta} = \sigma_L \circ T_L \circ \cdots \circ \sigma_1 \circ T_1 \quad (7.8)$$

donde cada transformación afín está dada por:

$$T_i(x) = W_i x + b_i \quad (7.9)$$

8 Función de Activación (Resumen)

Después de la convolución se aplica una función no lineal. Un ejemplo común es la función ReLU:

$$\phi(x) = \text{máx}(0, x) \quad (8.1)$$

La salida es:

$$Z = \phi(Y) \quad (8.2)$$

9 Pooling

El pooling reduce la dimensionalidad.

9.1. Max Pooling

$$Y(i, j) = \max_{(u, v) \in \mathcal{R}} X(i + u, j + v) \quad (9.1)$$

9.2. Average Pooling

$$Y(i, j) = \frac{1}{|\mathcal{R}|} \sum_{(u, v) \in \mathcal{R}} X(i + u, j + v) \quad (9.2)$$

10 Capa Totalmente Conectada

Se transforma el tensor en un vector:

$$X \in \mathbb{R}^{H \times W \times C} \rightarrow x \in \mathbb{R}^n \quad (10.1)$$

Luego:

$$y = Wx + b \quad (10.2)$$

donde $W \in \mathbb{R}^{m \times n}$ y $b \in \mathbb{R}^m$.

11 Función de Pérdida

11.1. Softmax

$$\hat{y}_i = \frac{e^{z_i}}{\sum_j e^{z_j}} \quad (11.1)$$

11.2. Entropía Cruzada

$$\mathcal{L} = - \sum_i y_i \log(\hat{y}_i) \quad (11.2)$$

12 Entrenamiento

El problema de optimización es:

$$\min_{\theta} \mathcal{L}(f(x; \theta), y) \quad (12.1)$$

Se utiliza descenso de gradiente:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L} \quad (12.2)$$

13 Propiedades Fundamentales

13.1. Conectividad Local

Cada neurona depende de una región pequeña de la entrada.

13.2. Compartición de Parámetros

El mismo kernel se aplica en toda la imagen.

13.3. Invarianza Traslacional

El modelo detecta patrones independientemente de su posición.

14 Flujo de una CNN

Imagen $\rightarrow$ Convolución $\rightarrow$ ReLU $\rightarrow$ Pooling $\rightarrow \dots \rightarrow$ Fully Connected $\rightarrow$ Softmax (14.1)

15 Interpretación Matemática

Una CNN puede verse como una función:

$$f : \mathbb{R}^{H \times W \times C} \rightarrow \mathbb{R}^k \quad (15.1)$$

compuesta por operadores lineales, no lineales y de reducción.

Las redes neuronales convolucionales son modelos altamente eficientes para el procesamiento de imágenes debido a la combinación de convoluciones, no linealidades y reducción de dimensionalidad, junto con propiedades estructurales como la compartición de parámetros y la conectividad local.

16